Name: Jesse BINJA
Roll Number: 202512173

ONLINE SHOPPING MANAGEMENT SYSTEM

UML Class Diagram Report

1. Introduction

This report presents the UML class diagram for the Online Shopping Management System. The diagram models the static structure of an online shopping platform by identifying its main classes, attributes, operations, and relationships. It provides a blueprint that can be used later during software design and implementation.

2. Project Overview and Objectives

The proposed system is intended to organise key online-shopping activities, including customer information management, product and stock management, shopping-cart management, order creation, and payment-record management.

- Identify the main classes required by the system.
- Specify the attributes and operations of each class.
- Represent associations and composition relationships between classes.
- Use multiplicities to show how many objects may participate in each relationship.
- Create a clear model that can guide future implementation.

3. Scope of the System

3.1 Included

- Customer registration and profile updates.
- Product details, prices, stock quantities, and availability checks.
- Adding and removing products from a shopping cart.
- Calculating cart totals and item subtotals.
- Creating, viewing, and cancelling orders.
- Recording and verifying payment information.

3.2 Excluded

- Physical delivery and shipment tracking.
- Supplier and warehouse management.
- Advanced financial reporting.
- Implementation of a complete production-ready website or mobile application.

4. Classes and Responsibilities

4.1 Customer

Represents a person using the platform.

Attributes: customerID, fullName, email, phone, address.

Operations: register(), updateProfile(), viewOrders().

4.2 ShoppingCart

Represents the active cart used to collect products before checkout.

Attributes: cartID, createDate.

Operations: addItem(), removeItem(), calculateTotal(), clearCart().

4.3 CartItem

Represents one product entry in a shopping cart.

Attributes shown in the diagram: productID, quantity, unitPrice.

Operations: updateQuantity(), calculateSubtotal().

4.4 Product

Represents a product that may be purchased.

Attributes: productID, name, description, price, stockQuantity.

Operations: updatePrice(), updateStock(), checkAvailability().

4.5 Order

Represents a purchase submitted by a customer.

Attributes: orderID, customerID, orderDate, status, totalAmount.

Operations: placeOrder(), cancelOrder(), calculateTotal().

4.6 OrderItem

Represents one product entry within an order.

Attributes shown in the diagram: productID, quantity, unitPrice.

Operation: calculateSubtotal().

4.7 Payment

Represents payment information associated with a purchase.

Attributes: payment, amount, paymentDate, paymentMethod, status.

Operations: processPayment(), verifyPayment().

5. Relationships and Multiplicities

The diagram uses associations to show links between independent classes and composition to show that cart items belong to a shopping cart and order items belong to an order.

- **Customer — ShoppingCart — Association:** The intended rule is that each shopping cart belongs to one customer, and a customer has zero or one active cart.
- **Customer — Order — Association:** Each order belongs to one customer. A customer may place zero or many orders.
- **ShoppingCart — CartItem — Composition:** A shopping cart contains zero or more cart items. Each cart item belongs to one shopping cart.
- **CartItem — Product — Association:** Each cart item refers to one product; the same product may appear in many cart items.
- **Order — OrderItem — Composition:** An order contains one or more order items. Each order item belongs to one order.
- **OrderItem — Product — Association:** Each order item refers to one product; a product may appear in many order items.
- **Order — Payment — Association (should be shown):** An order may have zero or more payment records, and each payment record should be linked to one order.

6. Notes for Improving the Diagram

Before submitting, check the diagram against the following modelling details. These are small corrections that will make the UML model clearer and more consistent.

- The Payment class appears disconnected in the supplied diagram. Draw an association between Order and Payment, with multiplicity 1 at the Order end and 0..* at the Payment end if multiple payment attempts are allowed.
- Check the Customer–Order multiplicities. The intended meaning is one Customer per Order and zero or many Orders per Customer. Put 1 beside Customer and 0..* beside Order.
- For Customer–ShoppingCart, use 1 beside Customer and 0..1 beside ShoppingCart if each customer can have at most one active cart.
- For ShoppingCart–CartItem, show composition with the filled diamond at the ShoppingCart end; use 1 beside ShoppingCart and 0..* beside CartItem.
- For Order–OrderItem, show composition with the filled diamond at the Order end; use 1 beside Order and 1..* beside OrderItem.
- The labels 'Linked to', 'Contains', and 'References' are understandable, but standard UML relationship lines and multiplicities should remain easy to read.
- Consider changing the Payment attribute 'payment: String' to 'paymentID: String' so it clearly identifies the payment record.
- The Order class includes customerID even though it is also associated with Customer. This is acceptable as an implementation-oriented identifier, but it may be redundant in a conceptual UML class diagram.

7. Conclusion

The UML class diagram models the principal parts of the Online Shopping Management System: customers, products, shopping carts, cart items, orders, order items, and payments. It defines the data each class stores and the operations each class supports. The associations and compositions describe how these objects work together. After the noted relationship and multiplicity checks are completed, the diagram can serve as a foundation for further system design.